JAWABAN TUGAS 3

PEMROGRAMAN JAVASCRIPT

Nama : Mu’min Hasanah

NIM : 2511102441314

Kelas : B

1. Analisis Perbedaan Parameter dan Argumen dalam Konteks Callback

a) Pada definisi fungsi prosesData(data, callback), data dan callback merupakan parameter, yaitu nama penampung nilai yang diterima fungsi. Pada pemanggilan prosesData("javascript", formatUpper), argumennya adalah nilai string "javascript" dan referensi fungsi formatUpper. Di dalam formatUpper, str juga merupakan parameter; ketika fungsi itu dipanggil, str menerima nilai "javascript".

b) prosesData termasuk Higher-Order Function karena menerima fungsi lain sebagai argumen melalui callback, lalu menjalankan fungsi tersebut. Sementara itu, formatUpper berperan sebagai Callback Function karena dikirim ke prosesData untuk dipanggil dari dalam prosesData. Callback tidak harus selalu berjalan secara asinkron; pada contoh ini callback dijalankan secara langsung.

c) Output yang tampil di console adalah JAVASCRIPT. Alurnya: prosesData menerima "javascript" sebagai data dan formatUpper sebagai callback. Baris return callback(data) kemudian memanggil formatUpper dengan data tersebut. Metode toUpperCase() mengubah huruf menjadi kapital, hasilnya dikembalikan melalui return, lalu console.log menampilkan hasil itu.

```
function prosesData(data, callback) {
    return callback(data);
}

const formatUpper = (str) => str.toUpperCase();

console.log(prosesData("javascript", formatUpper));

// Output: JAVASCRIPT
```

2. Analisis Penggunaan return dan Pure Function

a) Fungsi A bukan pure function karena arr.push(100) mengubah array yang diterima secara langsung. Perubahan terhadap data di luar hasil lokal fungsi ini merupakan side effect. Fungsi B tidak mengubah array asal; sintaks [...arr, 100] membuat array baru yang berisi seluruh elemen arr lalu menambahkan 100. Karena hasilnya dibuat tanpa memodifikasi input, Fungsi B lebih sesuai dengan prinsip pure function untuk contoh ini.

b) Pemanggilan berlangsung berurutan. Awalnya data bernilai [1, 2, 3]. Setelah tambahGlobal(data), elemen 100 ditambahkan ke array yang sama, sehingga data dan hasilA sama-sama menunjukkan [1, 2, 3, 100]. Kemudian tambahMurni(data) menerima data yang sudah berubah itu dan membuat array baru dengan tambahan 100 lagi. Jadi, hasilB bernilai [1, 2, 3, 100, 100].

```
function tambahGlobal(arr) {
    arr.push(100);
    return arr;
}

function tambahMurni(arr) {
    return [...arr, 100];
}

let data = [1, 2, 3];
let hasilA = tambahGlobal(data);
let hasilB = tambahMurni(data);

// data   = [1, 2, 3, 100]
// hasilA = [1, 2, 3, 100]
// hasilB = [1, 2, 3, 100, 100]
```

c) Fungsi B lebih direkomendasikan dalam pemrograman fungsional karena tidak mengubah data asal. Cara ini mengurangi efek samping dan membuat hasil fungsi lebih mudah diprediksi. Pernyataan return mengembalikan array baru dan langsung menghentikan eksekusi fungsi pada titik tersebut. Namun, perlu diperhatikan bahwa return saja tidak otomatis membuat fungsi menjadi pure; hal yang utama di sini adalah fungsi tidak memodifikasi input atau menimbulkan side effect.

3. Kombinasi Array Method dan Rest Parameter

a) Pada definisi function hitungRataRata(...angka), tanda ... berfungsi sebagai rest parameter: semua argumen yang diberikan akan dikumpulkan menjadi satu array bernama angka. Sebaliknya, pada pemanggilan hitungRataRata(...nilai), tanda ... adalah spread syntax yang membentangkan elemen array nilai menjadi argumen terpisah. Dengan demikian, fungsi menerima 80, 90, 70, dan 85, lalu rest parameter mengumpulkannya ke dalam array angka.

b) Metode reduce() memproses elemen array satu per satu untuk menghasilkan satu nilai akhir. Pada angka.reduce((a, b) => a + b, 0), nilai awal akumulator a adalah 0. Setiap langkah menambahkan elemen berikutnya b ke a, sehingga total akhirnya adalah 80 + 90 + 70 + 85 = 325.

c) Jumlah total 325 dibagi dengan panjang array, yaitu 4. Karena itu, nilai rata-ratanya adalah 325 / 4 = 81,25. Console JavaScript menuliskan desimal menggunakan titik, sehingga output yang tampil adalah 81.25.

```
function hitungRataRata(...angka) {
    let total = angka.reduce((a, b) => a + b, 0);
    return total / angka.length;
}

let nilai = [80, 90, 70, 85];
console.log(hitungRataRata(...nilai));

// Output: 81.25
```

4. Manipulasi Array dengan splice(), length, dan Iterasi

a) Untuk menghapus "JS" dan "React", gunakan teknologi.splice(2, 2). Parameter pertama, 2, adalah indeks awal penghapusan (indeks array dimulai dari 0), sedangkan parameter kedua, 2, menyatakan jumlah elemen yang dihapus. Setelah perintah ini, array menjadi ["HTML", "CSS", "Node.js"].

```
let teknologi = ["HTML", "CSS", "JS", "React", "Node.js"];
teknologi.splice(2, 2);
console.log(teknologi);
// Output: ["HTML", "CSS", "Node.js"]
```

b) Setelah penghapusan tersebut, teknologi.length = 1; mengubah panjang array menjadi satu elemen. Hanya "HTML" yang tersisa, sehingga array menjadi ["HTML"]. Elemen "CSS" dan "Node.js" yang masih ada setelah proses splice ikut terhapus dari array ketika panjangnya diperkecil. Elemen-elemen itu tidak lagi dapat diakses melalui array teknologi; jika tidak ada referensi lain yang menyimpannya, memorinya dapat dibersihkan oleh garbage collector.

```
teknologi.length = 1;
console.log(teknologi);
// Output: ["HTML"]
```

c) forEach() dapat menampilkan setiap elemen dengan menerima nilai elemen sebagai parameter pertama dan indeks sebagai parameter kedua. Karena kode dijalankan setelah length diatur menjadi 1, hanya "HTML" yang ditampilkan. Berbeda dari for biasa, forEach() menjalankan callback untuk setiap elemen yang tersedia secara otomatis. Namun, indeks tetap bisa diakses melalui parameter kedua. Pada for biasa, indeks dikelola sendiri, misalnya dengan variabel i, sehingga alur perulangan dan perubahan indeks dapat diatur lebih bebas.

```
teknologi.forEach((item, index) => {
    console.log(index + ": " + item);
});

// Output: 0: HTML
```

5. Refactoring: Dari Fungsi Konvensional ke Arrow Function dan IIFE

a) Fungsi yang berada di dalam buatCounter() disebut nested function karena didefinisikan di dalam fungsi lain. Fungsi tersebut tetap dapat mengakses count setelah buatCounter() selesai dijalankan karena terbentuk closure, yaitu fungsi beserta lingkungan leksikal yang dibutuhkannya. Variabel count tetap dipertahankan selama masih digunakan oleh fungsi yang dikembalikan. Setiap pemanggilan counter() menaikkan count satu angka lalu mengembalikan nilainya, sehingga output berturut-turut adalah 1 dan 2.

b) Berikut refactor menggunakan arrow function. Bentuk dan perilakunya tetap sama: buatCounter membuat count sekali, lalu mengembalikan fungsi yang dapat dipanggil berulang kali.

```
const buatCounter = () => {
    let count = 0;
    return () => ++count;
};

const counter = buatCounter();
console.log(counter()); // 1
console.log(counter()); // 2
```

c) IIFE (Immediately Invoked Function Expression) langsung dijalankan ketika ditulis dan sering digunakan untuk membuat scope lokal. IIFE yang hanya menjalankan proses lalu selesai kurang cocok untuk counter karena counter membutuhkan fungsi yang dapat dipanggil berkali-kali sambil menyimpan nilai count. Meski begitu, IIFE sebenarnya tetap bisa digunakan jika mengembalikan fungsi di dalamnya. Jadi, yang tidak cocok adalah IIFE yang hanya dieksekusi sekali tanpa menyediakan fungsi counter yang bisa dipanggil kembali; kebutuhan utama counter adalah closure yang mempertahankan state.

```
const counter = (() => { let count = 0; return () => ++count; })();
console.log(counter()); // 1
console.log(counter()); // 2
```